

CINEMAX

Manuale Tecnico

Laboratorio Interdisciplinare A - a.a. 2025/2026

Autori: Palomba, Riccardo, 764224; Carducci, Edoardo, 764215; Rosetti, Daniele, 767980; sede VA

Versione documento: 3.0 - Data: 30/08/2026

Sommario

1. Scopo e riferimenti	3
2. Struttura dell'applicazione.....	3
2.1 Panoramica dell'architettura	3
2.2 Flusso generale	3
3. Struttura del progetto	4
4. Modello a oggetti e responsabilità delle classi.....	4
4.1 Classe CineMax.....	4
4.2 Gerarchia degli utenti.....	4
4.3 Film	4
4.4 Proiezione	4
4.5 Posto	5
4.6 Prenotazione	5
4.7 ProgrammazioneCinema	5
5. Strutture dati.....	5
6. Flussi principali dell'applicazione	5
6.1 Avvio e caricamento.....	5
6.2 Autenticazione.....	6
6.3 Registrazione	6
7. Gestione delle proiezioni e ricerca	6
7.1 Inserimento di una proiezione	6
7.2 Modifica e cancellazione	6
7.3 Ricerca	6

<i>8. Gestione delle prenotazioni e dei posti</i>	7
8.1 Modifica della prenotazione	7
8.2 Eliminazione.....	7
L'eliminazione individua la prenotazione tramite il relativo ID e verifica che appartenga al cliente autenticato. Dopo aver richiesto una conferma, la prenotazione viene rimossa tramite ClienteRegistrato e i dati aggiornati vengono salvati nel file CSV. In caso di annullamento della conferma, l'operazione viene interrotta.	7
<i>9. Persistenza su file CSV</i>	7
9.2 Path relativi	7
9.3 Ciclo di vita dei dati.....	8
<i>10. Autenticazione e gestione delle password</i>	8
<i>11. Gestione degli input e degli errori</i>	8
<i>12. Scelte algoritmiche e complessità</i>	9
<i>13. Scelte architetturali e pattern</i>	9
<i>14. Javadoc e qualità del codice</i>	9
<i>15. Conclusioni</i>	10

1. Scopo e riferimenti

CineMax è un’applicazione Java da terminale per la gestione di un cinema. Il sistema consente di gestire utenti con ruoli differenti, consultare la programmazione, gestire le prenotazioni e amministrare le proiezioni. Il presente Manuale Tecnico descrive la struttura e il funzionamento interno dell’applicazione, con particolare attenzione a componenti, strutture dati, algoritmi, persistenza dei dati, gestione degli errori e documentazione.

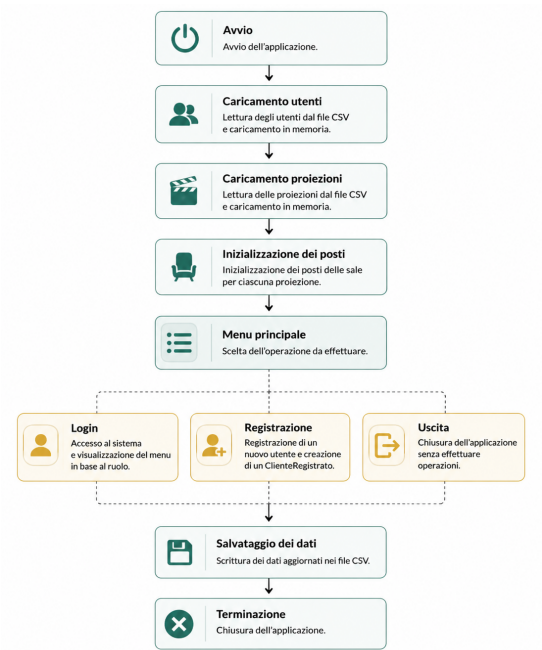
2. Struttura dell’applicazione

2.1 Panoramica dell’architettura

CineMax è un’applicazione Java da terminale organizzata nel package `cinemax`. La classe `CineMax` gestisce l’interazione con l’utente e il coordinamento delle operazioni, mentre i servizi si occupano della logica applicativa. I gestori gestiscono la lettura e la scrittura dei dati nei file CSV, mentre le classi del dominio rappresentano le principali entità del sistema, come utenti, film, proiezioni, posti e prenotazioni.

Area	Responsabilità	Componenti principali
Avvio e controllo	Menu, autenticazione, coordinamento dei flussi	CineMax,Menù,Sistema
Dominio	Rappresentazione di utenti, film, proiezioni, posti e prenotazioni	Utente, ClienteRegistrato, Proiezionista, Bigliettaio, Film, Proiezione, Posto, Prenotazione
Programmazione	Ricerca e gestione delle proiezioni	ProgrammazioneCinema
Persistenza	Lettura/scrittura dei dati	GestoreUtenti, GestoreProiezioni, GestorePrenotazioni
Sicurezza	Hash delle password	PasswordUtil

2.2 Flusso generale



3. Struttura del progetto

La configurazione documentata del progetto utilizza il package cinemax e separa il codice sorgente dai dati applicativi. I percorsi sono relativi alla directory principale del progetto.

Elemento	Funzione
src/cinemax/	Sorgenti Java dell'applicazione
data/utenti.csv	Utenti, credenziali hash e ruoli
data/proiezioni.csv	Programmazione e dati delle proiezioni
data/prenotazioni.csv	Prenotazioni persistite
docs/	Documentazione JavaDoc generata, se presente

4. Modello a oggetti e responsabilità delle classi

4.1 Classe CineMax

CineMax è il punto di coordinamento dell'applicazione. Mantiene lo Scanner per l'input, la collezione degli utenti, la lista delle proiezioni, la mappa dei posti per proiezione e il GestoreUtenti. I metodi di alto livello avviano l'applicazione, caricano/salvano i dati, mostrano i menu e delegano le operazioni alle classi di dominio.

```
private final Scanner scanner;
private final LinkedList<Utente> utenti;
private final List<Proiezione> proiezioni;
private final Map<String, LinkedList<Posto>> postiPerProiezione;
private final GestoreUtenti gestoreUtenti;
```

Nel codice documentato il metodo `avvia()` esegue la sequenza `caricaDati()` → ciclo del menu → `salvaDati()/salvaPrenotazioni()`, chiudendo infine lo Scanner.

4.2 Gerarchia degli utenti

Classe	Ruolo tecnico
Utente	Classe base per i dati comuni e l'autenticazione.
ClienteRegistrato	Specializzazione che rappresenta il cliente e le sue prenotazioni.
Proiezionista	Specializzazione responsabile della programmazione delle proiezioni.
Bigliettaio	Specializzazione dedicata alla consultazione/gestione delle prenotazioni al botteghino.

L'identificazione del comportamento in base al ruolo avviene dopo l'autenticazione. Nel flusso di login il programma individua l'utente e, in base al ruolo, apre il menu operativo corrispondente.

4.3 Film

Film rappresenta i dati descrittivi dell'opera cinematografica associata a una proiezione: titolo, genere, regista, anno, durata ed età minima. La classe separa le informazioni del film da quelle specifiche dell'evento di proiezione.

4.4 Proiezione

Proiezione rappresenta una singola programmazione di un film. Contiene un identificativo, il riferimento al Film, data e ora, prezzo del biglietto e una lista di posti. Nel codice analizzato ogni proiezione crea 200 posti, disposti su 20 file (A–T), con 10 posti per fila.

```

for (char fila = 'A'; fila <= 'T'; fila++) {
    for (int numero = 1; numero <= 10; numero++) {
        posti.add(new Posto(numero, fila));
    }
}

```

4.5 Posto

Posto rappresenta una posizione fisica nella sala, identificata dalla combinazione fila/numero. La sua istanza è associata alla proiezione e viene utilizzata per determinare la disponibilità durante una prenotazione.

4.6 Prenotazione

Prenotazione rappresenta l'associazione tra un cliente e una proiezione, con i dati necessari a identificare la prenotazione e i posti selezionati. Le operazioni di creazione, modifica ed eliminazione vengono eseguite nel contesto del cliente e della programmazione.

4.7 ProgrammazioneCinema

ProgrammazioneCinema incapsula la gestione della programmazione utilizzata per le ricerche. Il flusso applicativo costruisce la programmazione a partire dalla lista delle Proiezione e invoca il metodo di ricerca con lo Scanner.

5. Strutture dati

Struttura	Uso	Motivazione
LinkedList<Utente>	Elenco degli utenti	Inserimenti e scansioni semplici; dimensione tipicamente contenuta.
List<Proiezione> / ArrayList	Elenco della programmazione	Accesso sequenziale e gestione dinamica della collezione.
LinkedList<Posto>	Posti di una proiezione	Rappresentazione lineare semplice dei 200 posti.
Map<String, LinkedList<Posto>>	Indicizzazione posti per ID proiezione	Accesso diretto alla collezione di posti associata a un ID.
LocalDate	Date senza orario	Tipo standard Java per date di calendario.
LocalDateTime	Data e ora delle proiezioni	Rappresenta in modo tipizzato l'istante locale della programmazione.

Le strutture dati utilizzate sono state scelte in base al tipo di operazioni richieste dall'applicazione e alla natura dei dati da gestire. L'utilizzo di tipi specifici, come LocalDate e LocalDateTime, permette di rappresentare in modo corretto date e orari, evitando di gestirli direttamente come stringhe durante le operazioni interne. Le stringhe vengono utilizzate principalmente nelle fasi di input/output e nella lettura e scrittura dei file CSV, dove i dati devono essere convertiti nel formato previsto.

6. Flussi principali dell'applicazione

6.1 Avvio e caricamento

1. Crea le strutture dati principali.
2. Carica gli utenti tramite GestoreUtenti.
3. Legge le proiezioni tramite GestoreProiezioni.
4. Inizializza la struttura dei posti per ogni proiezione.

5. Carica le prenotazioni e ricostruisce le associazioni necessarie.
6. Avvia il menu principale.

6.2 Autenticazione

La procedura di login acquisisce username e password, calcola l'hash della password e ricerca l'utente corrispondente. Il confronto viene effettuato tra username e hash memorizzato; in caso di successo viene impostato lo stato di autenticazione e viene selezionato il menu in funzione del ruolo.

```
String password = hashPassword(leggiStringa("Password: "));
Utente utente = trovaUtente(username);

if (utente == null) {
    System.out.println("Username non trovato.");
    return;
}

if (!utente.getPassword().equals(password)) {
    System.out.println("Password errata.");
    return;
}
```

6.3 Registrazione

La registrazione costruisce un nuovo ClienteRegistrato dopo aver verificato l'unicità dello username. La password non viene mantenuta in chiaro ma trasformata in hash prima della memorizzazione.

7. Gestione delle proiezioni e ricerca

7.1 Inserimento di una proiezione

Il Proiezionista inserisce i dati del film e della programmazione. CineMax costruisce un Film, genera un nuovo ID di proiezione, costruisce la Proiezione, la aggiunge alla lista e inizializza i posti. La programmazione viene quindi persistita.

7.2 Modifica e cancellazione

La modifica della proiezione individua l'oggetto tramite ID e aggiorna la data/ora. La cancellazione individua la proiezione, richiede una conferma e la rimuove dalla collezione e dalla struttura dei posti indicizzata per ID, quindi aggiorna il CSV.

7.3 Ricerca

La ricerca viene delegata a ProgrammazioneCinema. I criteri documentati nel Manuale Utente comprendono titolo, genere, intervallo di date e costo massimo. I campi lasciati vuoti non restringono la ricerca.

```
ProgrammazioneCinema programmazione = new ProgrammazioneCinema();

for (Proiezione proiezione : proiezioni) {
    programmazione.aggiungiProiezione(proiezione);
}

return programmazione;
```

8. Gestione delle prenotazioni e dei posti

La prenotazione viene effettuata partendo da una proiezione selezionata e dalla disponibilità dei posti. Il programma verifica l'esistenza della proiezione, individua i posti richiesti e impedisce la selezione di posti inesistenti o già occupati. Le modifiche vengono propagate alle strutture in memoria e, al termine, ai file persistenti.

8.1 Modifica della prenotazione

Per modificare una prenotazione il programma individua la prenotazione tramite il suo ID e verifica che appartenga al cliente autenticato. Viene poi costruita una ProgrammazioneCinema con le proiezioni correnti e l'operazione viene delegata a ClienteRegistrato.

8.2 Eliminazione

L'eliminazione individua la prenotazione tramite il relativo ID e verifica che appartenga al cliente autenticato. Dopo aver richiesto una conferma, la prenotazione viene rimossa tramite ClienteRegistrato e i dati aggiornati vengono salvati nel file CSV. In caso di annullamento della conferma, l'operazione viene interrotta.

9. Persistenza su file CSV

9.1 Struttura e formato file

CineMax utilizza file **CSV locali** per memorizzare i dati in modo permanente tra una sessione e l'altra. La gestione della persistenza è affidata a componenti dedicati, che si occupano di **leggere i dati all'avvio e salvarli durante la chiusura dell'applicazione**, mantenendo separata questa attività dalla logica principale del sistema.

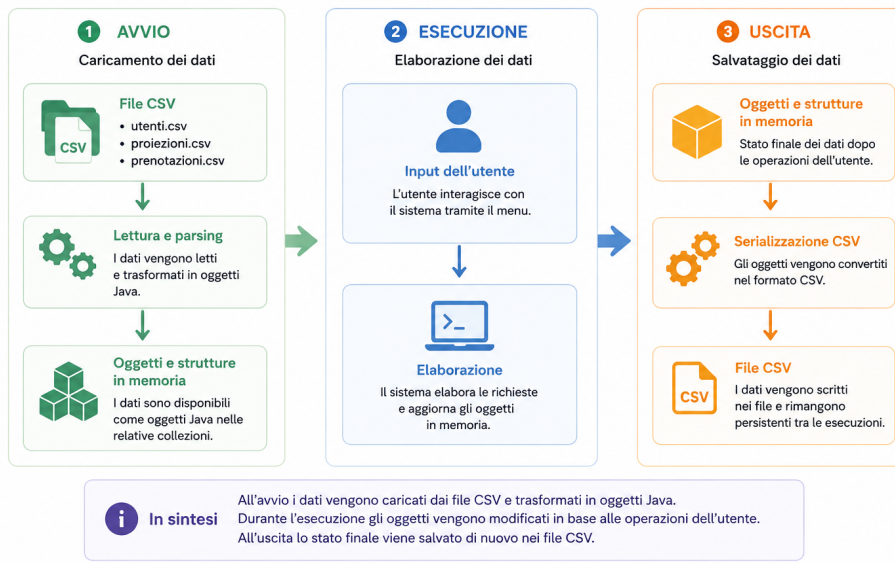
File	Gestore	Contenuto
data/utenti.csv	GestoreUtenti	Dati degli utenti, ruoli e password memorizzate tramite hash.
data/proiezioni.csv	GestoreProiezioni	Dati delle proiezioni e informazioni associate ai film.
data/prenotazioni.csv	GestorePrenotazioni / salvataggio prenotazioni	Prenotazioni salvate per mantenerle disponibili tra diverse esecuzioni.

9.2 Path relativi

I file CSV vengono raggiunti tramite percorsi relativi, ad esempio `data/proiezioni.csv`. Per questo motivo l'applicazione deve essere eseguita dalla directory principale del progetto, mantenendo invariata la struttura delle cartelle.

9.3 Ciclo di vita dei dati

Durante l'esecuzione, i dati seguono un ciclo ben definito che collega i file CSV alle strutture dati in memoria.



10. Autenticazione e gestione delle password

PasswordUtil gestisce la protezione delle password tramite **hash SHA-256**, utilizzando la classe MessageDigest. La password inserita dall'utente viene convertita in byte tramite codifica UTF-8, sottoposta alla funzione di hashing e infine trasformata in una stringa esadecimale.

```
MessageDigest digest = MessageDigest.getInstance("SHA-256");
byte[] bytes = digest.digest(password.getBytes(StandardCharsets.UTF_8));
```

La classe PasswordUtil è final e possiede un costruttore privato, poiché fornisce esclusivamente metodi statici. L'eventuale NoSuchAlgorithmException viene convertita in IllegalStateException, considerando l'assenza dell'algoritmo SHA-256 come una condizione non recuperabile durante l'esecuzione.

11. Gestione degli input e degli errori

L'interfaccia utente utilizza Scanner per acquisire i dati inseriti dall'utente. La lettura dei diversi tipi di input è centralizzata in appositi metodi, che gestiscono interi, double, stringhe, date e data/ora. Questa organizzazione evita la duplicazione del codice e permette di gestire gli input non validi in modo uniforme.

Caso	Gestione
Scelta menu non valida	Viene mostrato un messaggio di errore e l'utente ritorna al menu.
Input numerico non valido	L'input viene verificato e viene richiesto un nuovo valore.
Data/data-ora non valida	Il valore viene convertito tramite le classi java.time e, in caso di errore, viene richiesto un nuovo input.
Username già esistente	La richiesta viene ripetuta fino all'inserimento di uno username disponibile.
Proiezione inesistente	La ricerca viene effettuata tramite ID; se la proiezione non viene trovata, viene mostrato un messaggio e l'operazione viene interrotta.
Posto occupato/inesistente	L'operazione viene rifiutata e viene richiesta una scelta valida.
File non disponibile	L'errore di I/O viene gestito dai componenti responsabili della persistenza.

12. Scelte algoritmiche e complessità

Il progetto utilizza principalmente algoritmi semplici, basati sulla scansione delle strutture dati, adeguati alla quantità di dati prevista. Le scelte privilegiano **chiarezza, correttezza e semplicità di implementazione**, evitando ottimizzazioni non necessarie.

Operazione	Approccio	Costo indicativo
Ricerca utente per username	Scansione della LinkedList	$O(n)$
Ricerca proiezione per ID	Scansione della lista	$O(p)$
Inizializzazione posti	Doppio ciclo 20×10	$O(200) \rightarrow$ costante
Ricerca nei posti	Scansione della lista dei posti	$O(200) \rightarrow$ costante
Accesso ai posti per proiezione	HashMap per ID	$O(1)$ medio per lookup
Lettura CSV	Scansione delle righe	$O(r)$, con r numero di righe
Scrittura CSV	Scansione delle strutture	$O(r)$, con r numero di record

Le complessità indicate sono asintotiche e descrivono il costo delle principali operazioni in funzione della quantità di dati elaborati. Per le strutture a dimensione fissa, come i 200 posti di una sala, il costo viene considerato costante, poiché il numero di elementi non cresce con la dimensione dell'input.

13. Scelte architetturali e pattern

Non è stato adottato un pattern architetturale predefinito. La soluzione segue comunque una **separazione delle responsabilità**: CineMax coordina l'interazione con l'utente, le classi del dominio rappresentano le principali entità del sistema e i gestori dedicati si occupano della persistenza dei dati.

Scelta	Motivazione
Classi di dominio	Separano i dati del dominio dalle procedure di Input/Output.
Gestori CSV	Isolano la persistenza e riducono l'accoppiamento con i file.
ProgrammazioneCinema	Centralizza la logica di ricerca della programmazione.
Ereditarietà degli utenti	Permette di modellare i ruoli e specializzare il comportamento.
Map per i posti	Permette di associare rapidamente i posti all'ID della proiezione.
java.time	Permette di gestire date e orari tramite tipi specifici, evitando di trattarli direttamente come stringhe.

14. Javadoc e qualità del codice

La documentazione Javadoc è stata utilizzata per descrivere in modo strutturato le principali componenti del progetto. Il codice è stato quindi corredato da commenti Javadoc associati alle classi, ai metodi e agli attributi, con l'obiettivo di rendere più chiaro il funzionamento dell'applicazione.

Esempio di documentazione di una classe:

```
/**
 * Rappresenta una singola proiezione di un film.
 *
 * <p>Contiene film, data e ora, prezzo e posti
 * disponibili della sala.</p>
 *
 * @author Riccardo Palomba
 * @version 1.0
 */
```

15. Conclusioni

La soluzione CineMax è strutturata come un'applicazione Java CLI, basata sul modello a oggetti, con persistenza dei dati tramite file CSV e una separazione delle responsabilità tra i principali componenti del sistema. Le scelte effettuate privilegiano semplicità, leggibilità e coerenza con i vincoli del Laboratorio Interdisciplinare A. Per la consegna finale è fondamentale mantenere la **coerenza tra codice sorgente, JavaDoc, Manuale Utente e Manuale Tecnico**, così che la documentazione rifletta fedelmente il funzionamento dell'applicazione.